MAVUNO AGRIHUB | APPLICATION WALKTHROUGH

Cart Management

A source-grounded explanation of the cart from browser URL to Django view, session storage, rendered template, and JavaScript update.

Prepared October 2026
Scope: current implementation in the Ecommerce application

1. The mental model

The cart is a lightweight, anonymous, session-backed selection of products. It is not a database order and it does not reserve or deduct inventory. The browser sends cart operations to Django; Django validates them, updates the session, recalculates totals using current product records, and returns either a redirect or rendered HTML inside JSON.

```
Browser form / cart.js
        | POST (CSRF token; session cookie)
        v
Products.urls -> Products.views
        | read/write request.session["cart"]
        | query Product and current stock/prices
        v
cartContent.html fragment + count + messages (AJAX)
        | or redirect + Django messages (ordinary form post)
        v
Browser updates badge / cart UI
```

> **Cart value shape:** `{"<product primary key>": quantity}`. Example: `{"17": 2, "31": 1}`. Keys are normalized to strings and quantities to positive integers by `get_session_cart()`.

What is and is not included

- Implemented: add, increment/decrement, remove, cart page, session persistence, stock-aware quantity caps, current-price subtotal, and AJAX fragment refresh.
- Not implemented: checkout submission, order creation, payment, delivery calculation, inventory reservation/decrement, and cart-to-user account merging.

Primary source files: `Products/urls.py`, `Products/views.py`, `Products/cart_utils.py`, `Products/context_processors.py`, `static/scripts/cart.js`, and `Products/templates/Processes/`.

2. URL routing and HTTP operations

The project root URL configuration mounts the Products URLconf at `/products/`. Its namespace is `products`, so templates and Python use names such as `products:cart` rather than hardcoding those URLs.

Named route	Resolved URL pattern	Method	Purpose
`products:products`	GET /products/	GET	Active product catalog; supports search and category query parameters.
`products:cart`	GET /products/cart/	GET	Builds a fresh cart context and renders the cart page.
`products:cart_add`	POST /products/cart/add/<pk>/	POST only	Adds a quantity of an active product.
`products:cart_update`	POST /products/cart/update/<pk>/	POST only	Applies a one-unit adjustment (+1 or -1).
`products:cart_remove`	POST /products/cart/remove/<pk>/	POST only	Removes the product key from the session cart.
`products:detail`	GET /products/<pk>/	GET	Active product detail; also offers an Add to Cart form.

The cart-changing views use Django's `@require_POST`. The add/update views fetch an active product, so missing or inactive product IDs produce a 404. Remove is intentionally simpler: it discards the submitted key whether or not it was present.

Add form inputs and redirects

Catalog cards and product detail include a POST form, a CSRF token, and a hidden `next` field containing the current page. The quantity field is not displayed in these forms, so the view's default quantity is one. If a `next` URL is submitted, Django checks that it is safe for the current host; an unsafe external target is ignored. A normal non-AJAX add redirects back to that safe page, or to the cart if there is no safe target. Update/remove normally redirect to the cart.

3. Session storage and the global badge

How a visitor's cart survives requests

`SessionMiddleware` is enabled and precedes the views in `ecommerce/settings.py`. Django makes a session available as `request.session`. The cart views write the cart dictionary to the `cart` session key. The session middleware persists session changes and associates them with the visitor's session cookie. The settings do not override Django's session engine; with the installed sessions app, the normal Django database-backed session default applies.

The cart belongs to the browser session, not to an authenticated customer record. A different browser/profile has a different cart. There is no merge-on-login behavior in the cart code. Session expiry follows Django's configured/default session policy; there is no cart-specific expiration setting in this project settings file.

Input normalization

`get_session_cart(request)` handles malformed session values defensively. A non-dictionary becomes an empty cart. Each product key and quantity is converted to an integer; values that cannot convert, non-positive IDs, and non-positive quantities are discarded. The returned dictionary uses string product IDs. This cleans the in-memory view of the cart; it does not persist the cleaned value by itself. A cart view persists cleanup when it assigns the rebuilt dictionary back to the session.

Badge count is item units, not number of product lines

The `Products.context_processors.cart_summary` context processor reads the session and returns `cart_item_count = sum(cart.values())`. The shared `templates/layout.html` uses it to render the cart link's accessible label and optional badge on pages using that layout. The home page has a corresponding badge. Thus two units of one product count as two, not one line item.

> **Consistency detail:** the badge context processor only sums normalized session values; it does not query stock or product existence. The cart page's view does that validation and can prune stale products. Consequently a stale session value can be reflected in the global count until a cart-context-building request cleans it.

4. What the Django views do

Cart context: one authoritative calculation

`_cart_context()` is used by the cart page and AJAX responses. It:

1. Reads the normalized session cart.
2. Fetches matching active products in one queryset and prefetches their images.
3. Drops missing, inactive, or zero-stock products.
4. Caps quantities that exceed current stock and queues an informational message.
5. Builds each line total from the current unit price times quantity; sums the subtotal and unit count.
6. Writes the rebuilt cart back when it changed, and returns item rows, subtotal, and item count.

Prices are `Decimal` values from the current Product rows, so arithmetic avoids binary floating-point rounding. The cart stores IDs and quantities only: it does not freeze a price, product name, or image at add time.

Add operation

- Requires an active product; rejects no-stock items.
- Parses posted `quantity` (default `1`); rejects invalid or less-than-one values.
- Adds to any existing quantity, then caps the stored amount at current stock.
- Reports success, or informs the shopper if the request exceeded stock.

Update and remove operations

- Update accepts only `adjustment=-1` or `adjustment=1`; other values are rejected with an error message.
- Increment is blocked at current stock. If the product becomes out of stock, update removes it.
- Decrementing from one to zero removes the product line. Otherwise it stores the adjusted quantity.
- Remove pops the product key and writes the session cart. Repeating removal is safe because the key may be absent.

`MessageMiddleware` and the messages context processor provide user feedback. For AJAX, the helper reads/consumes pending messages and serializes each message's text and tags into the response.

5. AJAX: how cart.js keeps the page in sync

The shared layout loads `static/scripts/cart.js`. It uses delegated event handling for form submissions, so it continues to work after the cart fragment is replaced. Only forms marked `data-cart-operation` are intercepted.

1. Prevent the browser's default form navigation and temporarily disable the submit button.
2. POST the form's action and `FormData`, with same-origin credentials and `X-Requested-With: XMLHttpRequest`. The CSRF token is already a hidden form field.
3. Parse the JSON response. The server returns `cart_html`, `item_count`, and `messages`.
4. Update every cart link's `data-count`, accessible `aria-label`, and badge (adding or removing the badge as needed).
5. Update the cart page's count label, replace `#cart-content` with the returned Django-rendered fragment, and reinitialize Lucide icons if available.
6. Show each server message in a short-lived accessible notice. Re-enable the button in `finally`.

The server recognizes AJAX by the exact `X-Requested-With: XMLHttpRequest` header. If a cart form is posted without it, views use the normal redirect path instead. If fetch/JSON parsing or the request fails, JavaScript displays a local error notice; the server's regular Django messages remain the fallback for non-AJAX navigation.

> **Server-rendered fragment, not client-calculated totals:** JavaScript does not independently calculate prices, stock, or line totals. Each successful request returns HTML generated from the current server-side cart context, keeping business rules in Django.

Accessibility affordances include the live cart count, cart content region, feedback region, accessible labels on quantity buttons, and status badge. The asynchronous flow depends on JavaScript; the forms still have a redirect-based server path without it.

6. Templates and the shopper journey

Template	Responsibility in the cart flow
`Products/templates/Cartegories/catalog.html`	Lists products and provides an Add to Cart POST form per product, with CSRF and a safe-return candidate.
`Products/templates/Processes/productDetail.html`	Shows product/stock details and an Add to Cart form. The button is disabled in the UI when stock is zero.
`Products/templates/Processes/cartPage.html`	Extends the shared layout, shows the cart heading/count, and places the reusable fragment inside `#cart-content`.
`Products/templates/Processes/partials/cartContent.html`	Renders each item, quantity +/- controls, remove, subtotal summary, or empty-cart state. This same fragment is used for full page rendering and AJAX refreshes.
`templates/layout.html`	Displays the global cart link/badge and loads the shared cart JavaScript.

Typical shopper flow

1. Open `/products/` or a product detail page.
2. Choose Add to Cart. The POST form includes the product ID in the URL, the CSRF token, and a hidden return location.
3. JavaScript submits asynchronously. Django checks current stock and stores the quantity under the visitor's session.
4. The page badge is refreshed without a full navigation. On the cart page, the same response also replaces item rows and summary.
5. Use plus/minus to adjust by one or Remove to delete a line. If the browser lacks JavaScript, POST redirects still allow the server-rendered workflow.

Cart summary semantics

The summary shows total units and subtotal. Delivery is explicitly “Calculated at checkout”; no delivery amount is computed here. The subtotal is not a final payable total, and taxes/fees are not modeled in this cart fragment.

7. Stock, safety, and operational limits

Stock is checked but not reserved

Adding and increasing quantities are capped by the stock quantity read for that request. Rendering the cart also reconciles quantities with stock and removes products that are now inactive, missing, or out of stock. This is helpful for display correctness, but it is not a transaction that reserves inventory. Another purchase/stock change can still happen after the cart is rendered.

Request protections

- Cart mutations are POST-only; the forms include Django CSRF tokens and the CSRF middleware is enabled.
- Product IDs are resolved against active products for add/update.
- Quantity input is parsed and validated; stock caps are applied server-side, not trusted from the UI.
- The optional return URL is host-validated before redirecting.
- AJAX requests use same-origin credentials; the session cookie ties the operation to the visitor.

Checkout is deliberately not wired

The cart template renders a disabled “Checkout coming soon” button and says order placement/payment will be available once checkout is connected. There is no cart checkout URL in `Products/urls.py`. The project's root URL configuration also leaves Orders and Payments includes commented out. `Products/templates/Processes/checkOut.html` is empty. Therefore this cart currently ends at review/editing; it does not create an Order or initiate a Payment.

> **Important for the next step:** proceeding from this cart to “products ordered” requires a separate checkout/order workflow. That work must revalidate stock and price at submission time and should define inventory reservation/decrement, delivery, payment, and failure behavior rather than treating this session cart as an order.

8. Tests and source map

The Products test suite contains focused coverage for the cart, including adding a product and showing the summary; AJAX add and quantity responses; plus/minus/remove actions; stock caps and invalid adjustments; and pruning unavailable or missing products. These tests are in `Products/tests.py` under `ProductManagementTests`.

Concern	Source
URL names and patterns	`Products/urls.py`; mounted by `ecommerce/urls.py`.
Request handling, rules, totals, responses	`Products/views.py`.
Session cart normalization	`Products/cart_utils.py`.
Shared badge count	`Products/context_processors.py`, configured in `ecommerce/settings.py`.
Session, message, CSRF configuration	`ecommerce/settings.py`.
Product current stock and price fields	`Products/models.py`.
AJAX form delegation and visual refresh	`static/scripts/cart.js`.
Cart screen and reusable fragment	`Products/templates/Processes/cartPage.html` and `Products/templates/Processes/partials/cartContent.html`.
Add entry points	`Products/templates/Cartegories/catalog.html`, `Products/templates/Processes/productDetail.html`, and the home product cards in `templates/index.html`.

This guide describes the inspected workspace implementation, not a proposed future design. The project README contains no additional cart documentation.